Software Maintenance and Evolution:

Types of Software, Lehman's 'Laws' and management & quality of maintenance

Topics covered

- Software Evolution Strategies
- *E*-, and *S*-type Software
- Lehman's 'Laws'
- Appreciate the cost of maintenance
- The management of maintenance
- Qualities in maintenance

Maintenance & Evolution

- Software Maintenance
  - consists of the activities required to keep a software system operational and responsive after it is accepted and placed into production

- Software Evolution
  - a continuous change from a lesser, simpler, or worse state to a higher or better state

Software Evolution

> "Software development does not stop when a system is delivered but continues throughout the lifetime of the system"
> – Sommerville, 2004

- The system changes relate to changing needs – business and user
- The system evolves continuously throughout its lifetime
- The process is a spiral process involving requirements, design & implementation throughout the lifetime of the system

Software evolution strategies

- Software maintenance
  - Adaptive, Corrective, Perfective, Preventative
- Software re-engineering
  - No new functionality is added to the system but it is restructured and reorganised to facilitate future changes
- Maintenance and re-engineering may be applied separately or together

E- and *S*-type Software

***E*-type** software is a solution to real-world problem, used and embedded in a real-world domain. It is judged as satisfactory by its functionality, quality factors (usability, etc.), performance, changeability, and so on.

***S*-type** software has the sole criterion of being mathematically correct with respect to a fixed and constant specification.

Lehman's 'laws' (i – iv)

I 1974	Continuing Change	*E*-type systems must be continually adapted else they become progressively less satisfactory in use
II 1974	Increasing Complexity	As an *E*-type system is evolved its complexity increases unless work is done to maintain or reduce it.
III 1974	Self Regulation	Global *E*-type system evolution processes are self-regulating. [System attributes such as size, time between releases and the number of reported errors are approximately invariant for each system release.]
IV 1978	Conservation of Organisational Stability	Unless feedback mechanisms are appropriately adjusted, average effective global activity rate in an evolving *E*-type system tends to remain constant over product lifetime.

Lehman's 'laws' (v – viii)

V 1978	Conservation of Familiarity	In general, the incremental growth and long term growth rate of *E*-type systems tend to decline.
VI 1991	Continuing Growth	The functional capability of *E*-type systems must be continually increased to maintain user satisfaction over the system lifetime.
VII 1996	Declining Quality	Unless rigorously adapted to take into account changes in the operational environment, the quality of *E*-type systems will appear to be declining.
VIII 1996	Feedback System (Recognised 1971, formulated 1996)	*E*-type evolution processes are multi-level, multi-loop, multi-agent feedback systems.

Lehman, Ramil, Wernick, Perry and Turski, "Metrics and Laws of Software Evolution—The Nineties View," *Proceedings of the 4th International Software Metrics Symposium (METRICS '97),* IEEE, November 1997, pp 20-32 (http://www.ece.utexas.edu/~perry/work/papers/feast1.pdf)

One way to … Service a Maintenance Request 1

1. **Be prepared to keep required metrics. Include..**
   - **… lines of code added**
   - **… lines of code changed**
   - **… time taken: 1. preparation 2. design 3. code 4. test**
2. **Ensure that the request has been approved**
3. **Understand the problem thoroughly**
   - **reproduce the problem**
     - **otherwise get clarification**
4. **Classify the MR as *repair* or *enhancement***
5. **Decide whether the implementation requires a redesign at a higher level**
   - **if so, consider batching with other MR's**
6. **Design the required modification (i.e., incorporate the change)**

One way to … Service a Maintenance Request 2

7. Plan transition from current design
8. Assess change's impact throughout the application
   - small changes can have major impact!
9. Implement the changes
10. Perform unit testing on the changed parts
11. Perform regression testing
    - ensure changes haven't compromised existing capabilities
12. Perform system testing with new capabilities
13. Update the configuration, requirement, design and test documentation

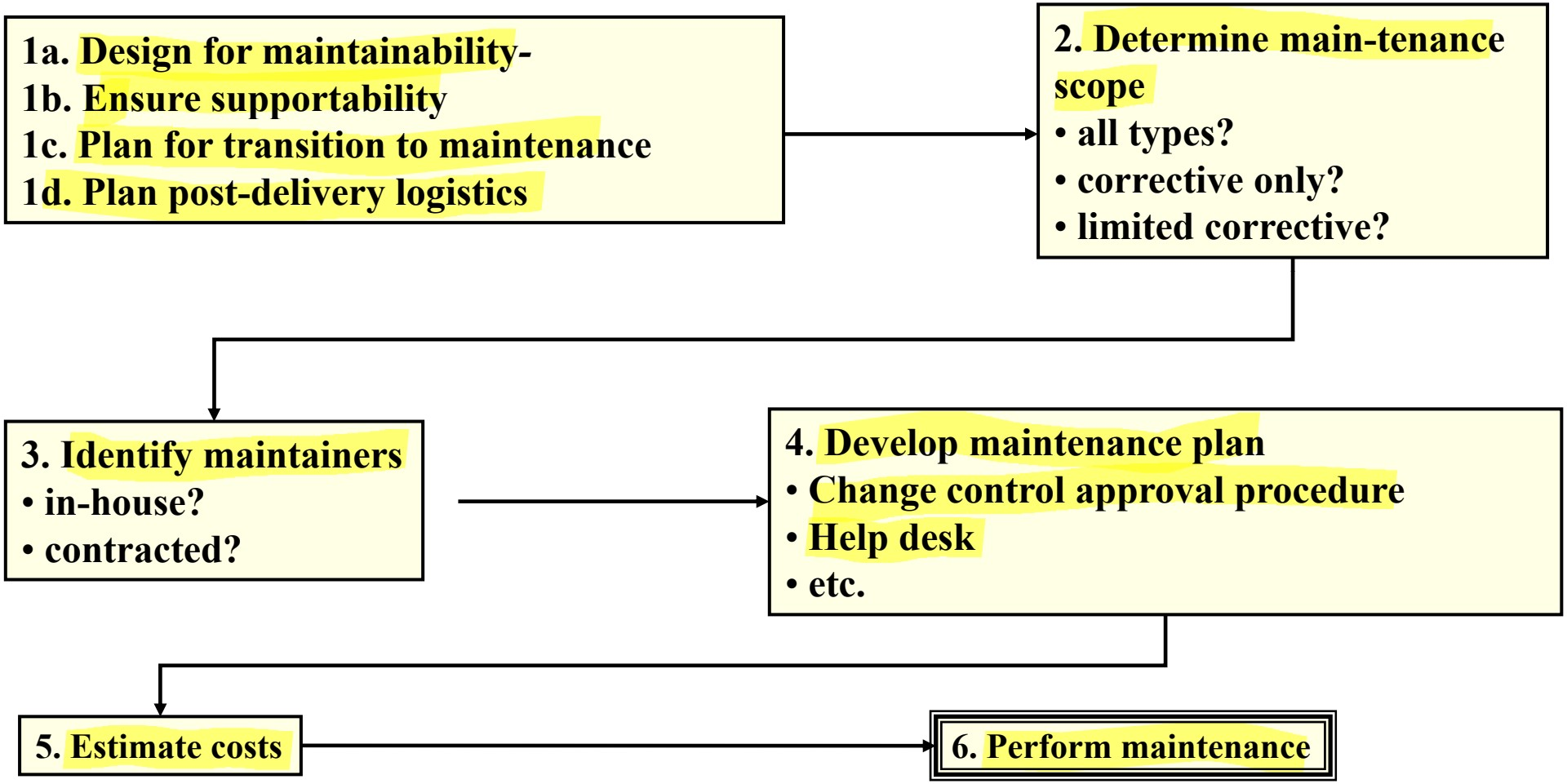
Software Maintenance Issues

- ***Management***
  - **Return on investment hard to define**
- ***Process***
  - **Extensive coordination required to handle stream of Maintenance Requests**
- ***Technical***
  - **Covering full impact of changes**
  - **Testing very expensive compared with the utility of each change**
    - **focused tests ideal but expensive**
    - **regression testing still required**

Estimating the Cost of Servicing a Maintenance Request

Activity	Estimate (person-days)	Activity	Estimate (person-days)
1. Understand the problem and identify the functions that must be modified or added.	2 - 5	6. Compile and integrate into baseline	2 - 3
2. Design the changes	1 - 4	7. Test functionality of changes	2 - 4
3. Perform impact analysis	1 - 4	8. Perform regression testing	2 - 4
4. Implement changes in source code	1 - 4	9. Release new baseline and report results	1
5. Change SRS, SDD, STP, configuration status	2 - 6	**TOTAL**	**14 - 35**

RoadMap to Establish Maintenance



Example of *Corrective* Maintenance Request

Maintenance Request 78

The computations that ensue when the player changes the value of a quality, are supposed to keep the total invariant, but they do not. For example, if the qualities are *strength* = 10, *patience* = 0.8 and *endurance* = 0.8 (sum = 11.6), and the player adjusts *strength* to 11, then the result is *strength* = 11, *patience* = 0 and *endurance* = 0, which do not sum to 11.6.

Example of *Perfective* Maintenance Request

Maintenance Request 162

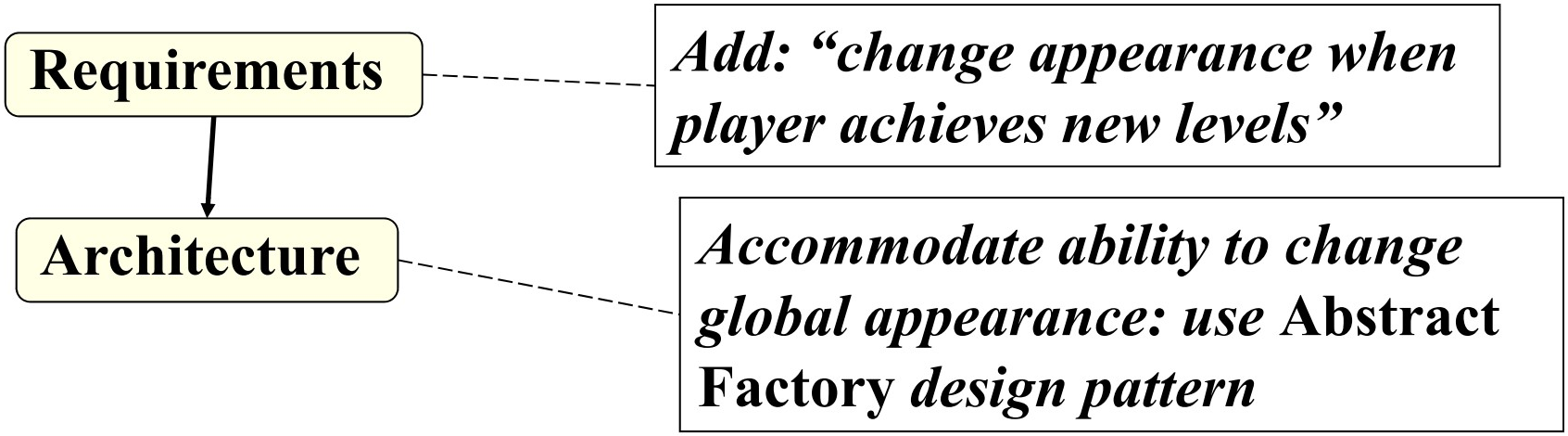
Modify *Encounter* so that the game begins with areas and connections a coordinated style.

Example of *Perfective* Maintenance Request

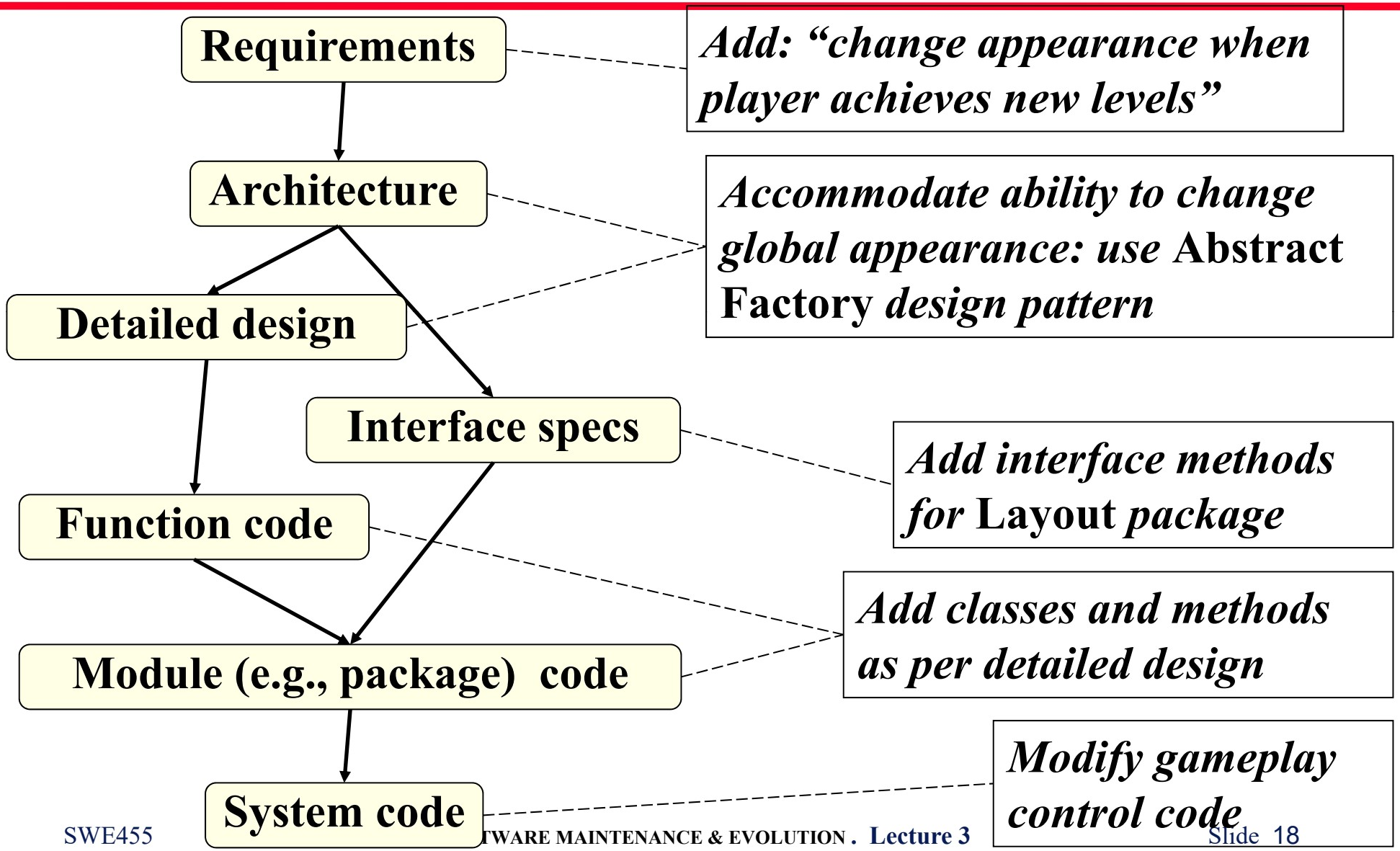
Maintenance Request 162

Modify *Encounter* so that the game begins with areas and connections in a coordinated style. When the player achieves level 2 status, all areas and connections are displayed in an enhanced coordinated style, which is special to level *2* etc. The art department will provide the required images.

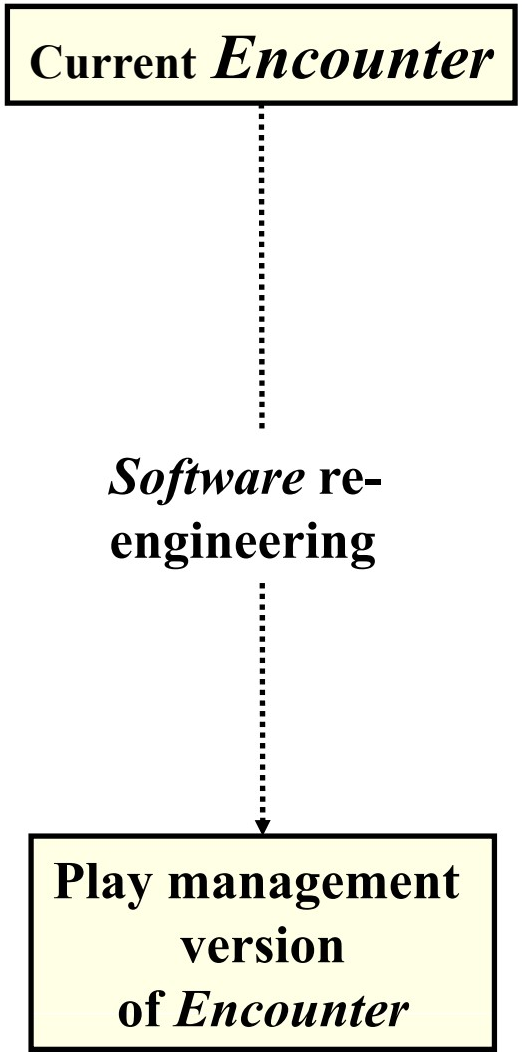
Impact of MR #162



Impact of MR #162

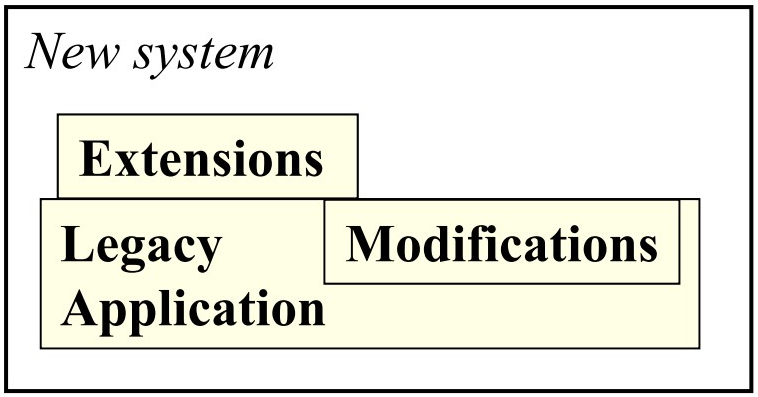


Re-engineering Management Training; Re-engineering *Encounter* to Conform



Legacy Architectures

Incorporation



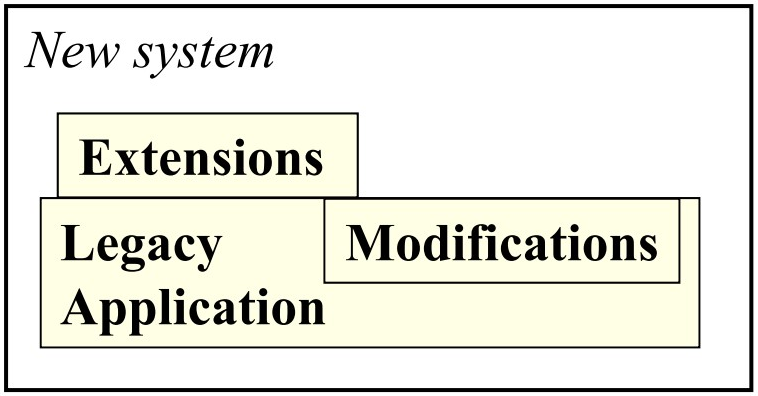
(fig i)0

Encapsulation

Legacy Architectures

uses...

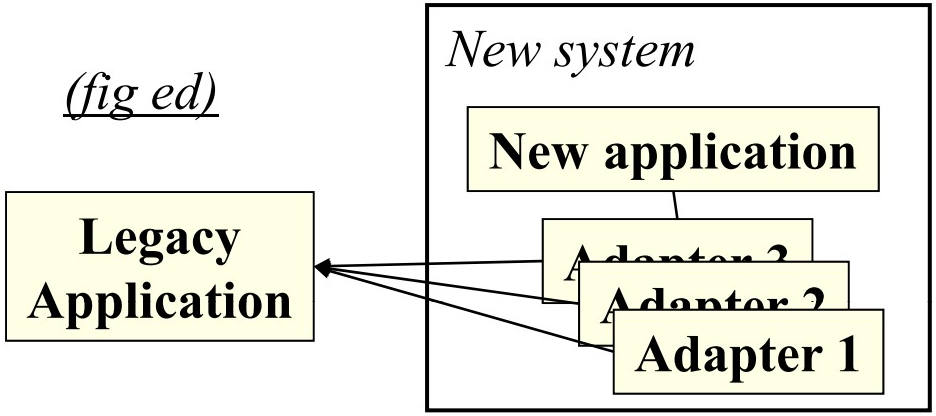
Incorporation



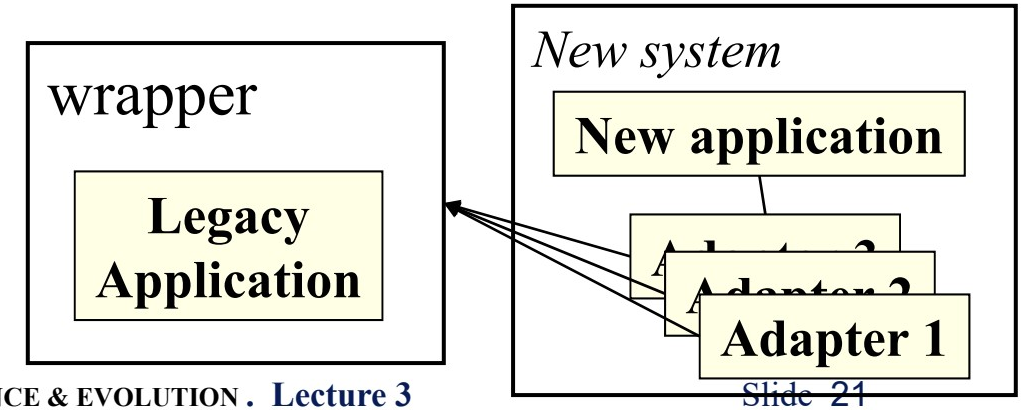
(fig i)

Encapsulation

(fig ed)



(fig ew)



1. Problem identification

1.1 Input 1.2 Process

1.3 Control 1.4 Output

1.5 Quality factors

1.6 Metrics

2. Analysis

2.1 Input

2.2 Process

2.2.1 Feasibility analysis

2.2.2 Detailed analysis

2.3-2.6 Control, Output, Quality factors, Metrics.

3. Design

3.1-3.6 Input, Process, Control, Output, Quality factors, Metrics.

IEEE 840-1994
"Software Maintenance"

1. Problem identification

1.1 Input 1.2 Process

1.3 Control 1.4 Output

1.5 Quality factors

1.6 Metrics

2. Analysis

2.1 Input

2.2 Process

2.2.1 Feasibility analysis

2.2.2 Detailed analysis

2.3-2.6 Control, Output, Quality factors, Metrics.

3. Design

3.1-3.6 Input, Process, Control, Output, Quality factors, Metrics.

IEEE 840-1994 "Software Maintenance" Table of Contents

4. Implementation

4.1 Input

4.2 Process

4.2.1 Coding and & testing

4.2.3 Risk analysis & review

4.2.4 Test-readiness review

4.3-4.6 Control, Output, Quality factors, Metrics.

5. System test

5.1-5.6 Input, Process, Control, Output, Quality factors, Metrics. **6. Acceptance test**

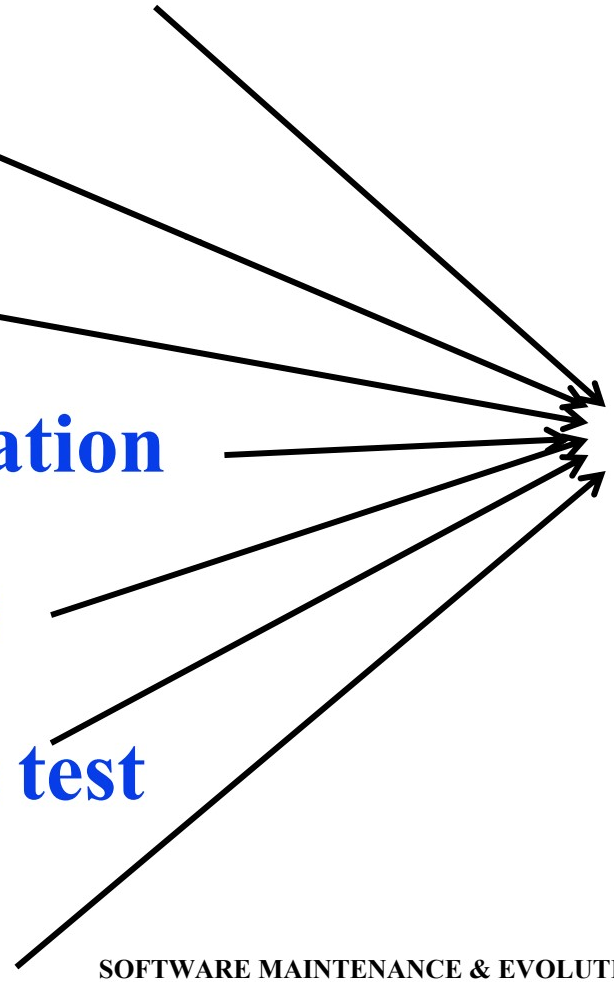
6.1-6.6 Input, Process, Control, Output, Quality factors, Metrics. **7. Delivery**

7.1-7.6 Input, Process, Control, Output, Quality factors, Metrics.

Five Attributes of Each Maintenance Step (IEEE)

Step

1. **Problem identification**
2. **Analysis**
3. **Design**
4. **Implementation**
5. **System test**
6. **Acceptance test**
7. **Delivery**



Attributes

a. Input life cycle arti-facts for this step
b. Process required for this step
c. How the process is controlled
d. Output life cycle artifacts
e. Process quality factors involved
f. Metrics for this step

IEEE 1219-1992
Maintenance phase 1: *Problem Identification*

a. Input	•The Maintenance Request (MR)
b. Process	•Assign change number •Classify by type and severity etc. •Accept or reject change •Make preliminary cost estimate •Prioritize
c. Control	•Identify MR uniquely •Enter MR into repository
d. Output	•Validated MR
e. Selected quality factors	•Clarity of the MR •Correctness of the MR (e.g., type)
f. Selected metrics	•Number of omissions in the MR •Number of MR submissions to date •Number of duplicate MR's •Time expected to confirm the problem

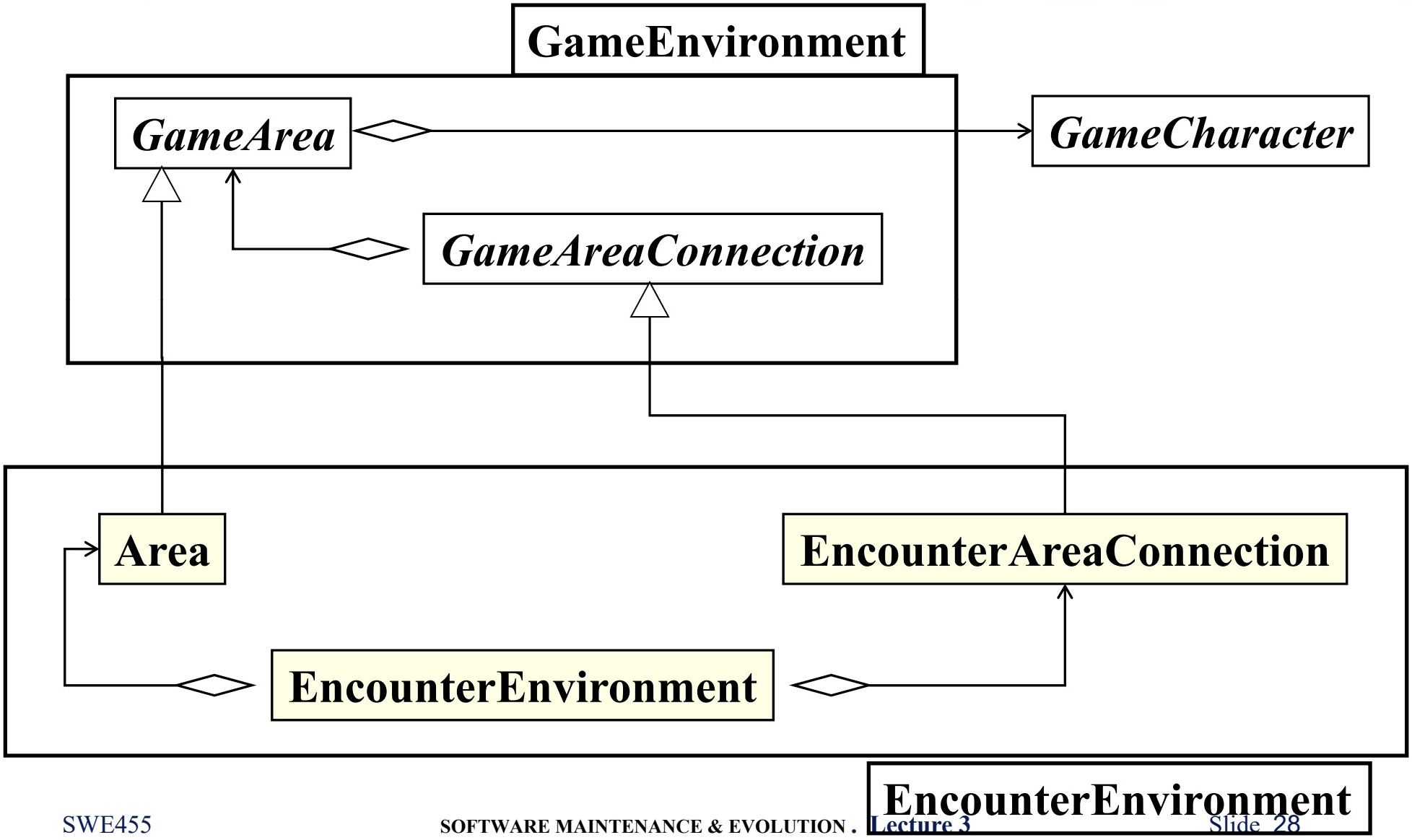
IEEE 1219-1992
Maintenance phase 2: *Problem Analysis*

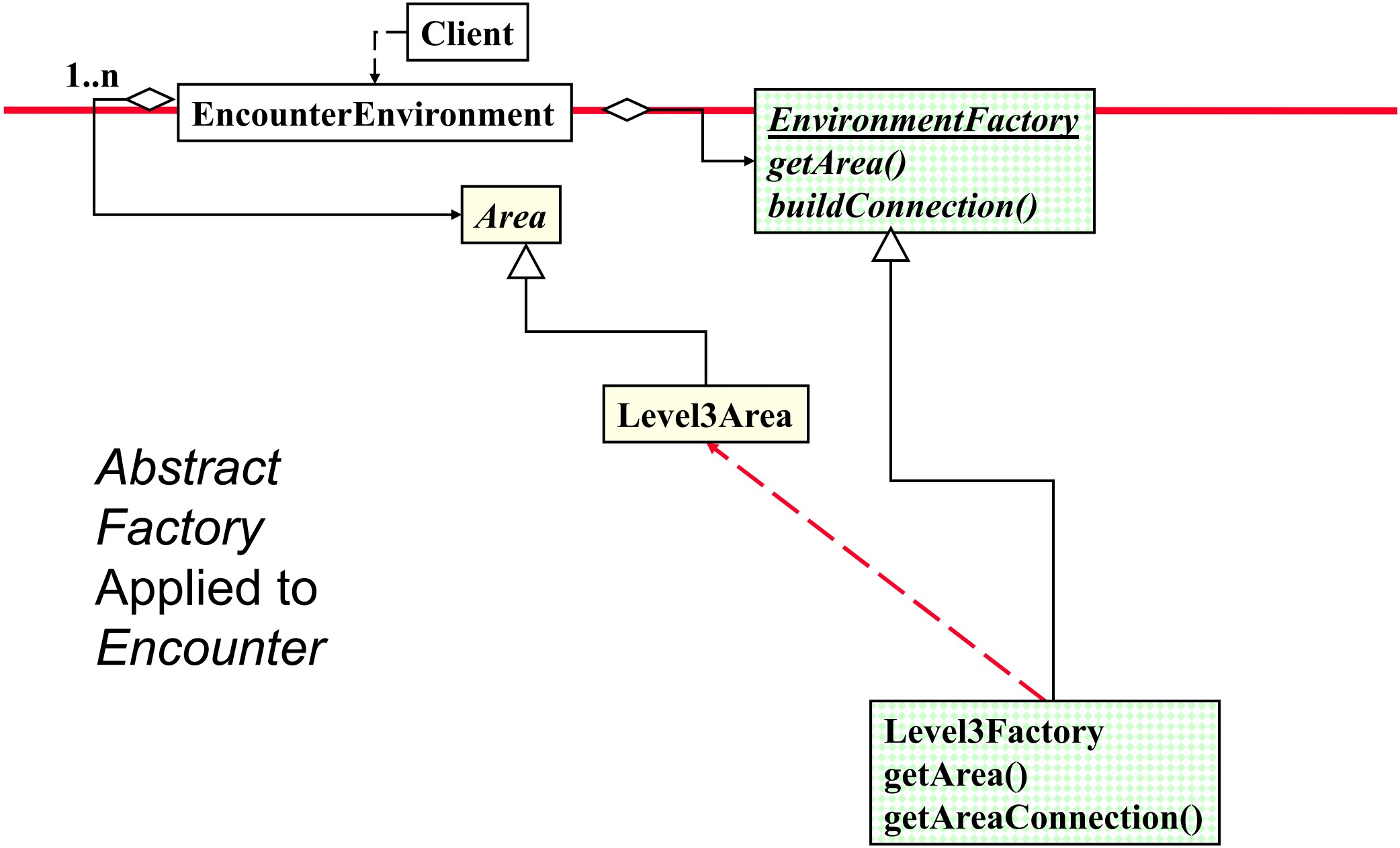
a. Input	•Original project documentation •Validated MR from the identification phase
b. Process	•Study feasibility of the MR •Investigate impact of the MR •Perform detailed analysis of the work required •Refine the MR description
c. Control	•Conduct technical review •Verify … …test strategy appropriate …documentation updated •Identify safety and security issues
d. Output	•Feasibility report •Detailed analysis report, including impact •Updated requirements •Preliminary modification list •Implementation plan •Test strategy
e. Selected quality factors	•Comprehensibility of the analysis
f. Selected metrics	•Number of requirements that must be changed •Effort (required to analyze the MR) •Elapsed time

IEEE 1219-1992
Maintenance phase 3: *Design*

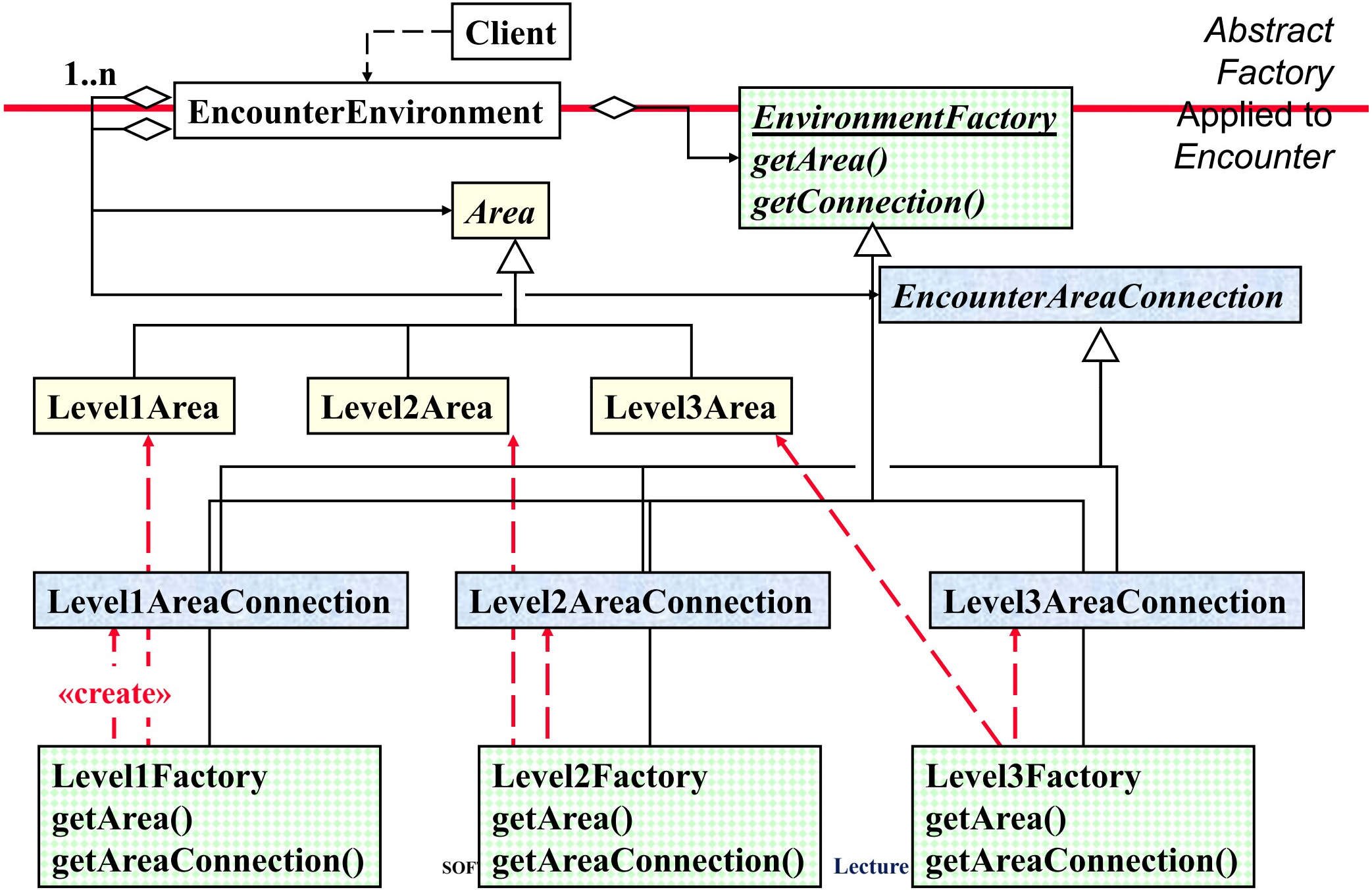
a. Input	•Original project documentation •Analysis from the previous phase
b. Process	•Create test cases •Revise … …requirements …implementation plan
c. Control	•Verify design •Inspect design and test cases
d. Output	•Revised … …modification list …detailed analysis …implementation plan •Updated … …design baseline …test plans
e. Selected quality factors	•Flexibility (of the design) •Traceability •Reusability •Comprehensibility
f. Selected metrics	•Effort in person-hours •Elapsed time •Number of applications of the change

EncounterEnvironment Package (Before Modification)

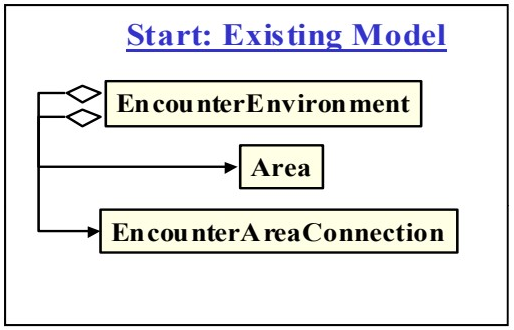


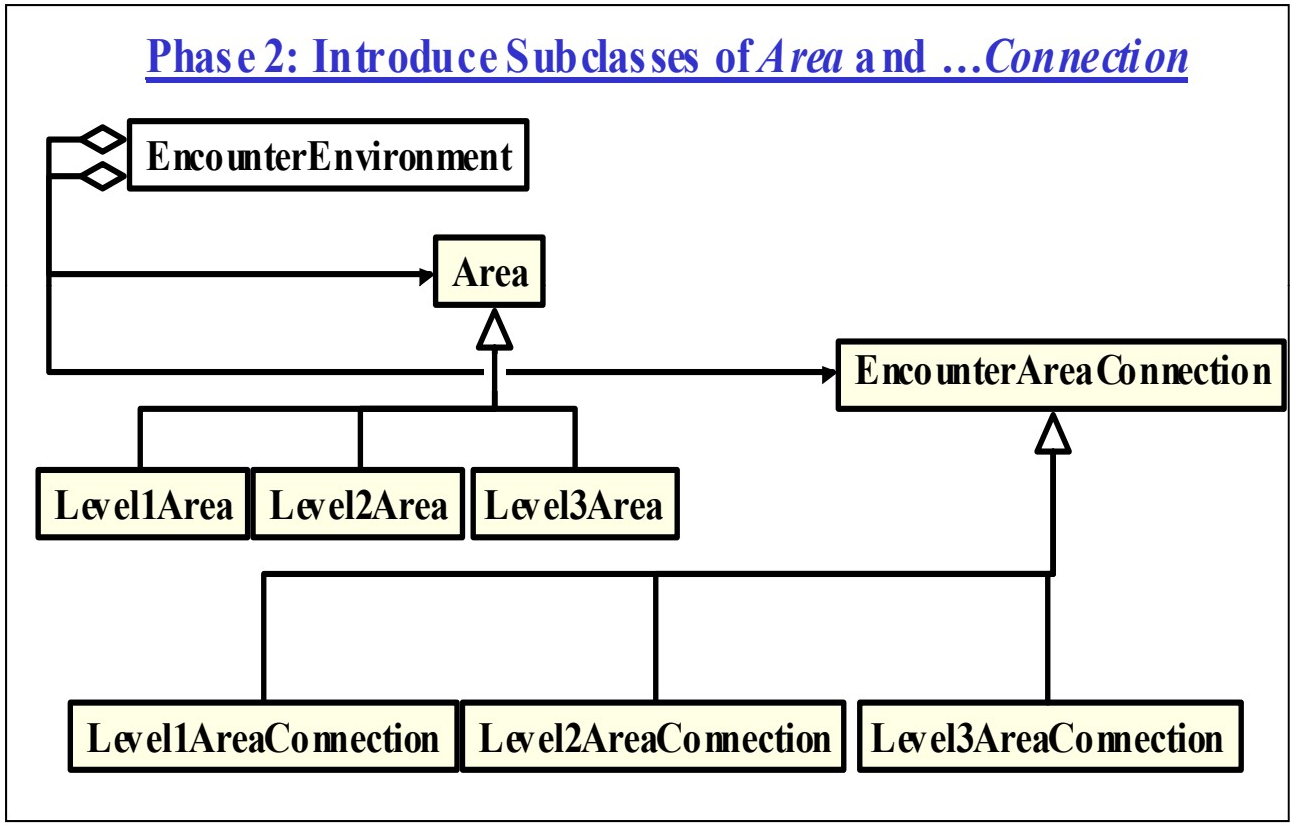


Abstract Factory Applied to *Encounter*

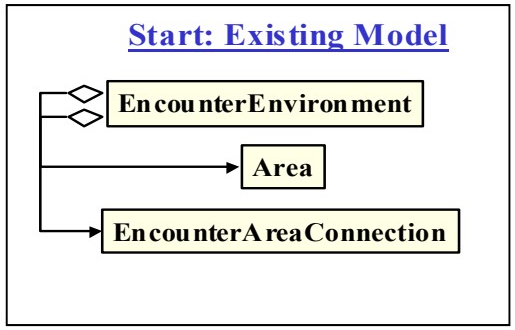

Client
1..n
EncounterEnvironment
Area
EnvironmentFactory
getArea()
getConnection()
Abstract Factory Applied to Encounter
EncounterAreaConnection
Level1Area
Level2Area
Level3Area
Level1AreaConnection
Level2AreaConnection
Level3AreaConnection
«create»
Level1Factory
getArea()
getAreaConnection()
Level2Factory
getArea()
getAreaConnection()
Level3Factory
getArea()
getAreaConnection()

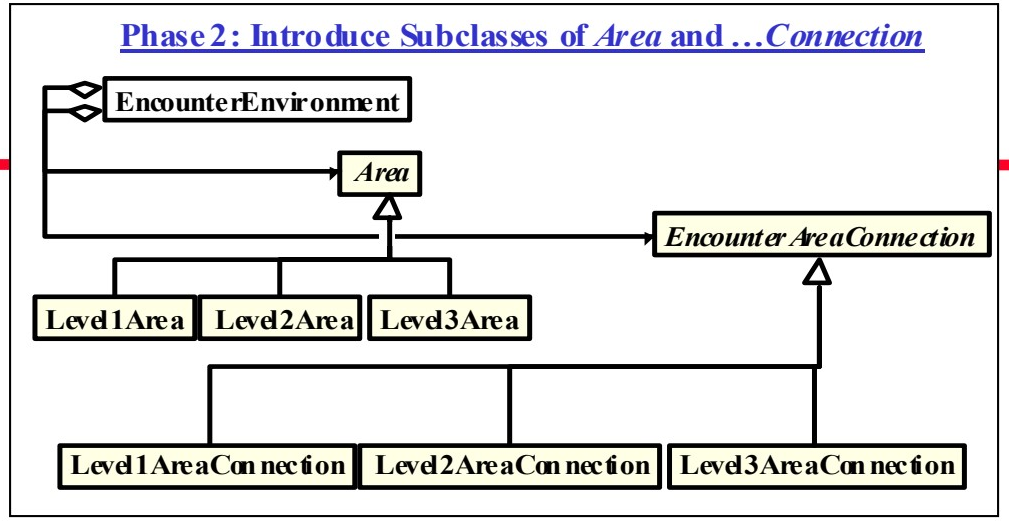
Migration Plan for Level-based Graphics

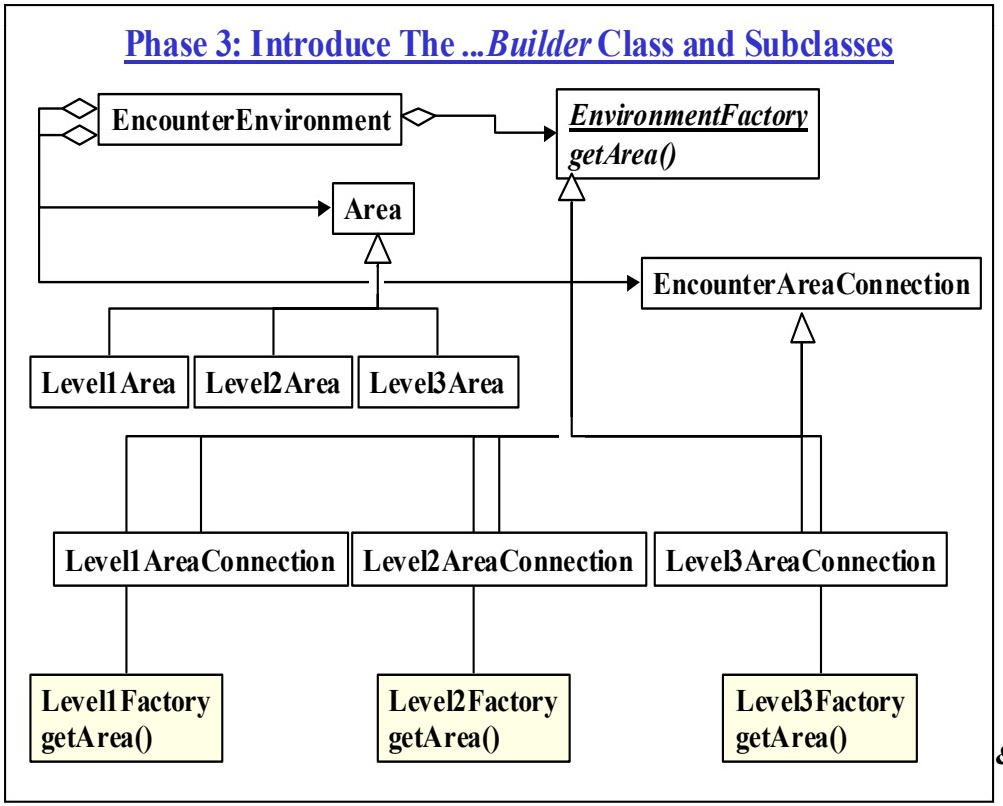




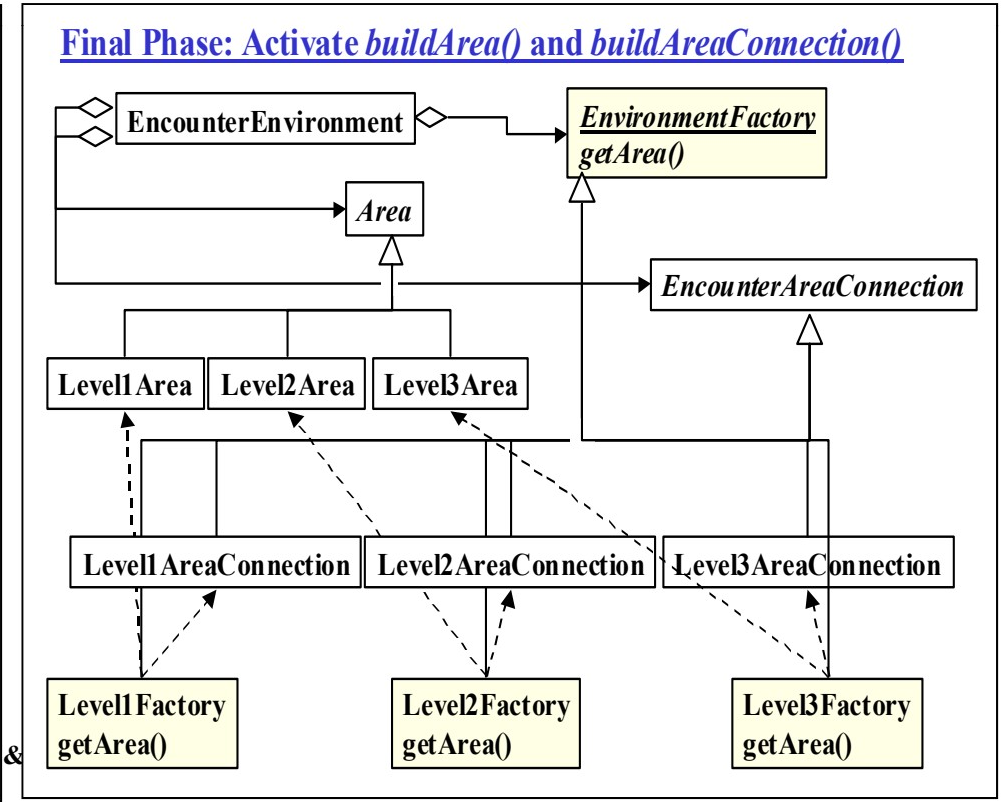
Migration Plan for Level-based Graphics







&



IEEE 1219-1992
Maintenance phase 4: *Implementation*

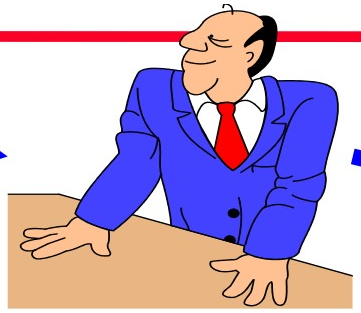
a. Input	•Original source code •Original project documentation •Detailed design from previous phase
b. Process	•Make code changes and additions •Perform unit tests •Review readiness for system testing
c. Control	•Inspect code •Verify CM control of new code Traceability of new code
d. Output	•Updated … …software …unit test reports …user documents
e. Selected quality factors	•Flexibility •Traceability •Comprehensibility •Maintainability •Reliability
f. Selected metrics	•Lines of code •Error rate

The management of maintenance

A Typical Maintenance Flow

Customer

nominal path



Help desk

Written MR's

Proposed M. R.'s

Change control board

Approved M. R.'s

Maintenance engineer

Current source & documentation

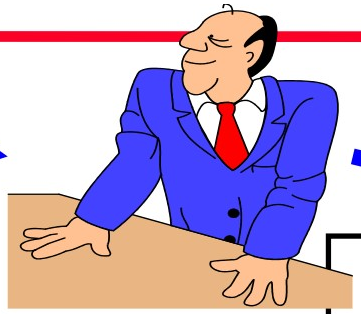
Modified source & documentation

A Typical Maintenance Flow

Customer

nominal path

Maintenance manager



Help desk

Written MR's

Marketing

Proposed M. R.'s

Maintenance engineer

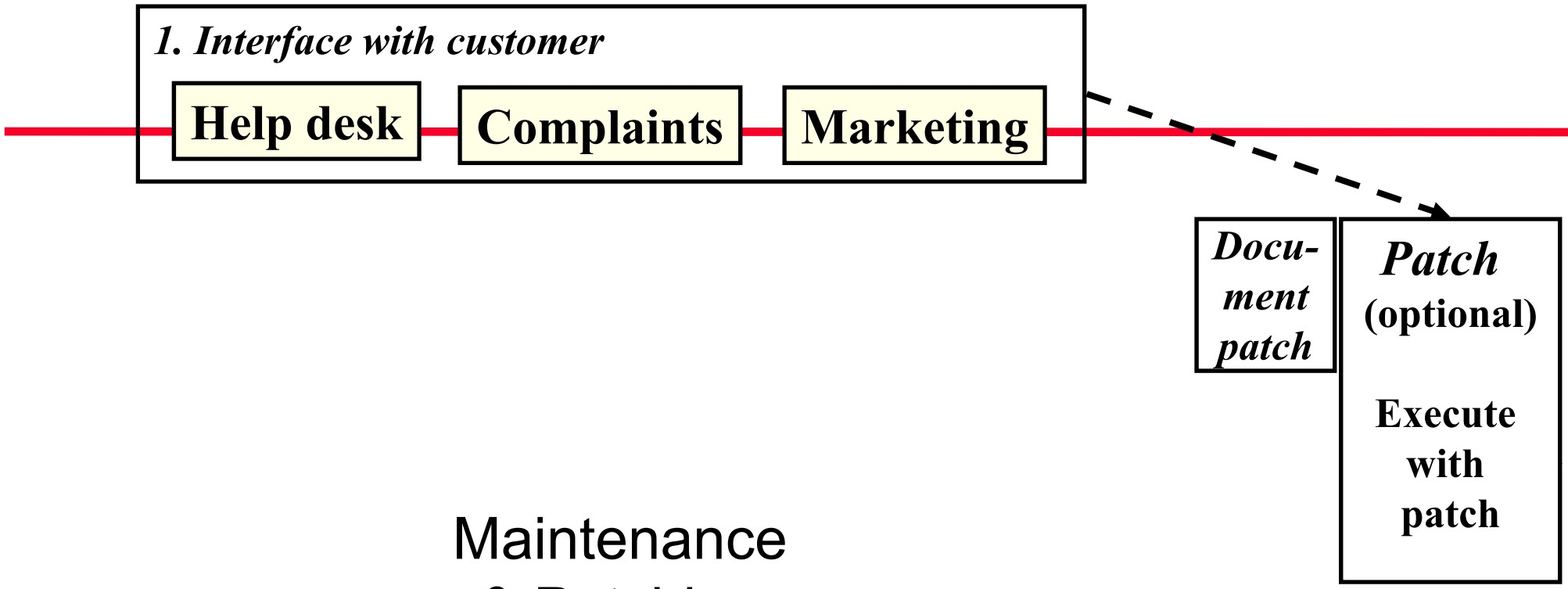
Current source & documentation

Approved M. R.'s

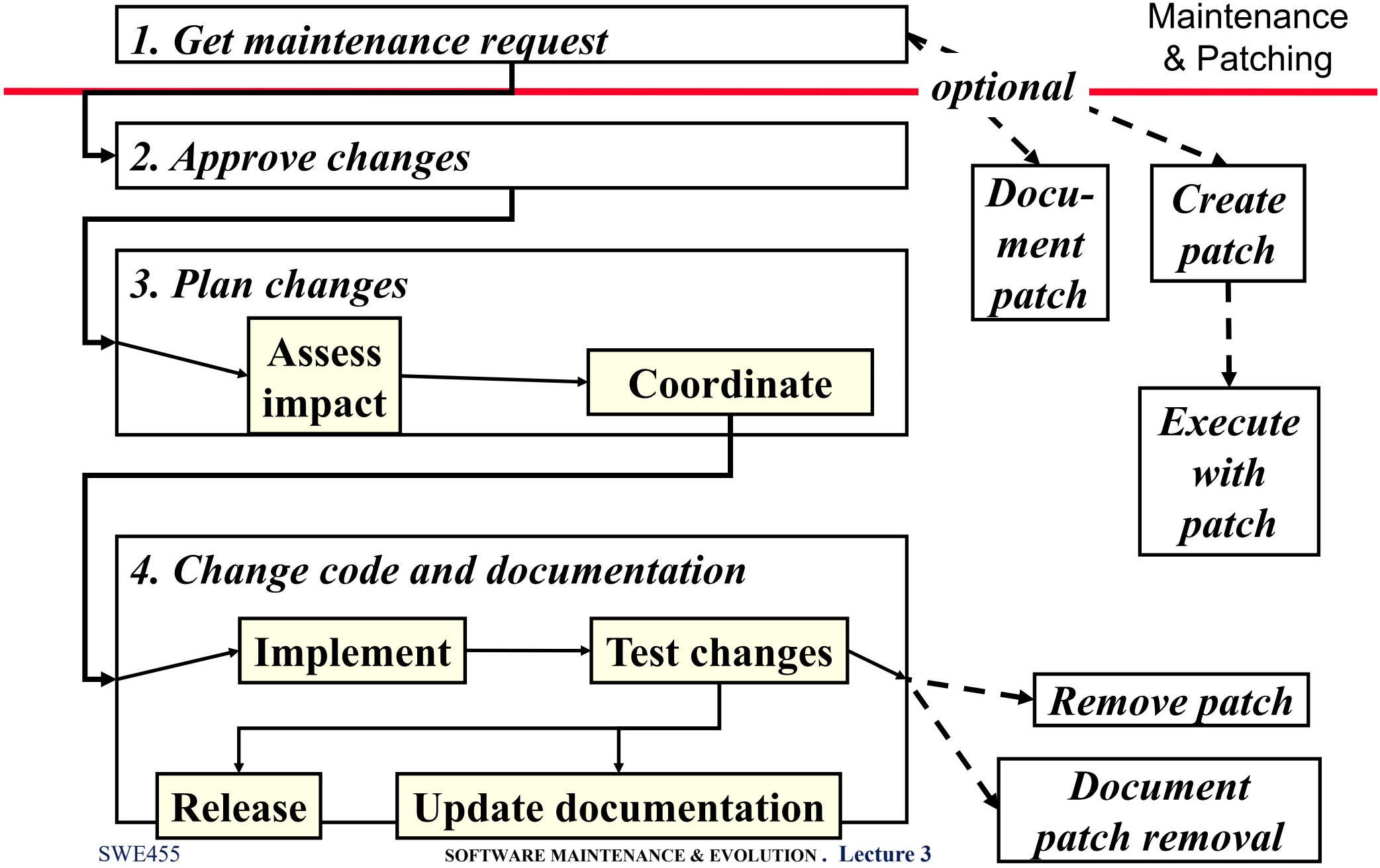
Change control board

Modified source & documentation

Rejected MR's



Maintenance & Patching


1. Get maintenance request
2. Approve changes
3. Plan changes
Assess impact
Coordinate
4. Change code and documentation
Implement
Test changes
Release
Update documentation
Maintenance & Patching
optional
Docu-ment patch
Create patch
Execute with patch
Remove patch
Document patch removal

Maintenance Patches

advantages

- Keeps customers satisfied in the short run
- Enables continued operation and testing without repeated prevalence of the defect
- Avoids masking other defects
- Enables test of fix

disadvantages

- Duplicates work
  - patch *and* final fix both implemented
- Sometimes never replaced
  - proper fix deferred forever!
- Complicates final fix
  - must remove
- Complicates documentation process

Ranked Problems in Maintenance

1. Changing priorities
2. Testing methods
3. Performance measurement
3. Incomplete or non-existent system documentation
5. Adapting to changing business requirements
6. Backlog size
7. Measurement of contributions
8. Low morale due to lack of recognition or respect
9. Lack of personnel, especially experienced
10. Lack of maintenance methodology, standards, procedures and tools . . . .

Top priority . . . Examples of Changing Priorities

. . . at release :

- ***Make this most bug-free game on the market***
  - **action: eliminate as many defects as possible**

. . . two months after release:

- ***Add more features than our leading competitor***
  - **action: add enhancements rapidly**

. . . six months after release:

- ***Reduce spiraling cost of maintenance***
  - **action: eliminate most severe defects only**

Qualities in maintenance

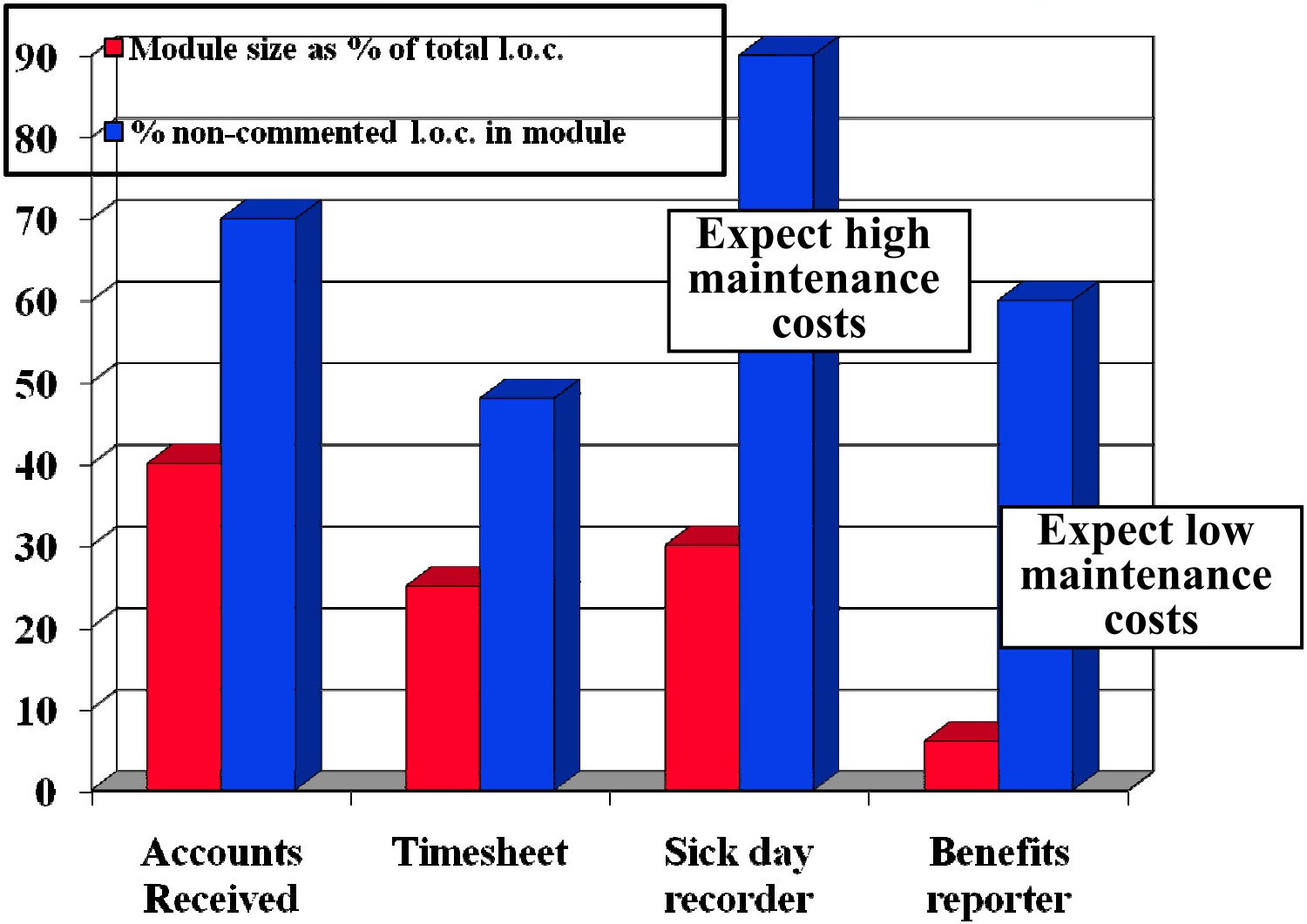
Maintenance Metrics

- **Number of lines of code under maintenance**
- **Person-months to perform various maintenance tasks**
- **Defect count**

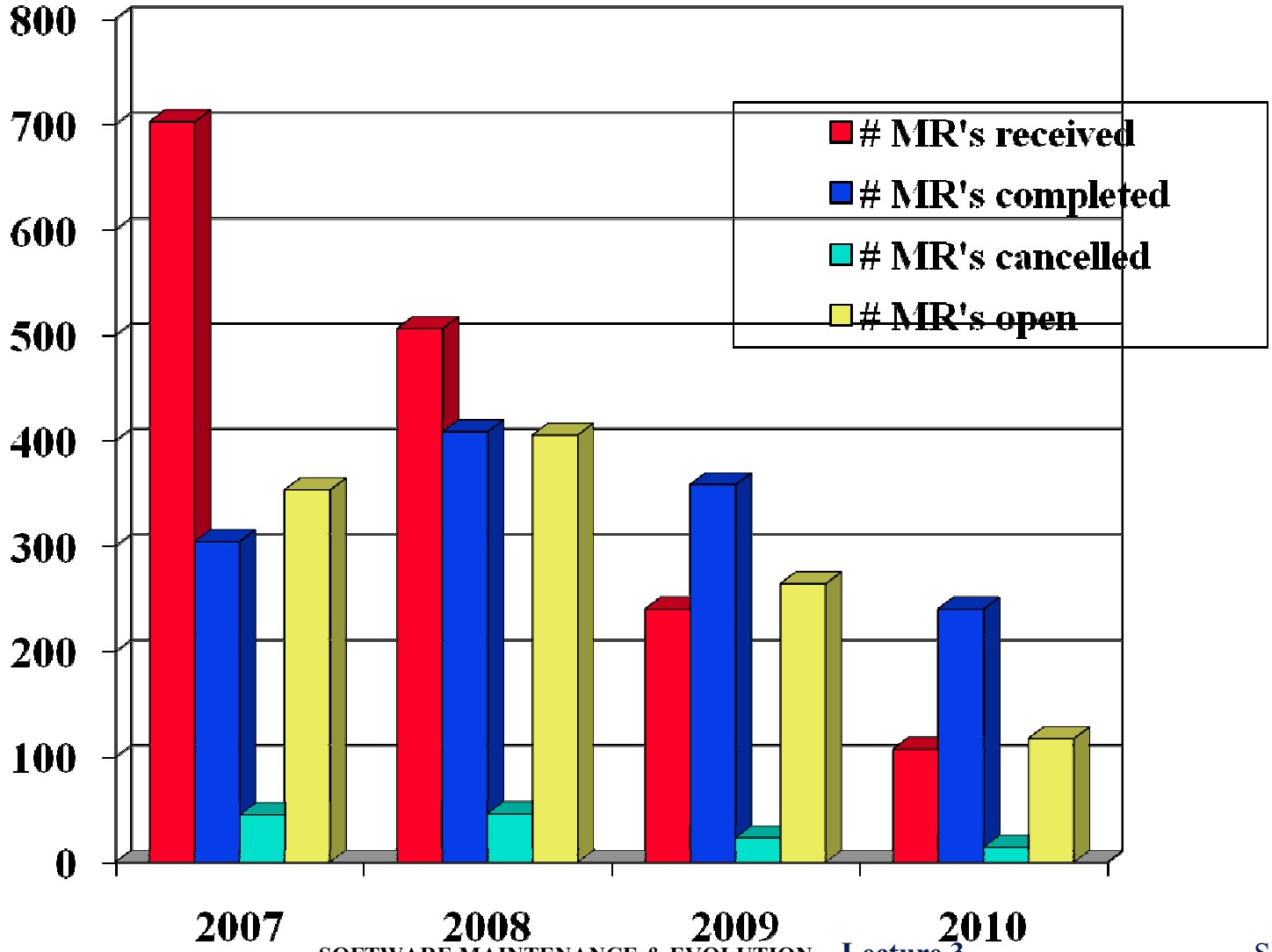
Maintenance Metrics Classified by Goal

Goal	Question	Selected Corresponding Metrics Note: The numbered metrics are from the IEEE.
Maximize customer satisfaction	*How many problems are affecting the customer?*	•(1) Fault density •(30) Mean time to failure •Break / fix ratio [Number of defects introduced by maintenance actions] / [Number of defects repaired]
	How long does it take to fix a problem?	•Fault closure Average time required to correct a defect, from start of correction work. •Fault open duration Average time from defect detection to validated correction.
	Where are the bottlenecks?	•Staff utilization per task type: Average person-months to (a) detect each defect and (b) repair each defect. •Computer utilization Average time / CPU time per defect.
Optimize effort and schedule	*Where are resources being used?*	Effort and time spent, per defect and per severity category … o… planning o… reproducing customer finding o… reporting error o… repairing o… enhancing
Minimize defects (continue focused development-type testing)	*Where are defects most likely to be found?*	•(13) Number of entries and exits per module •(16) Cyclotomic complexity

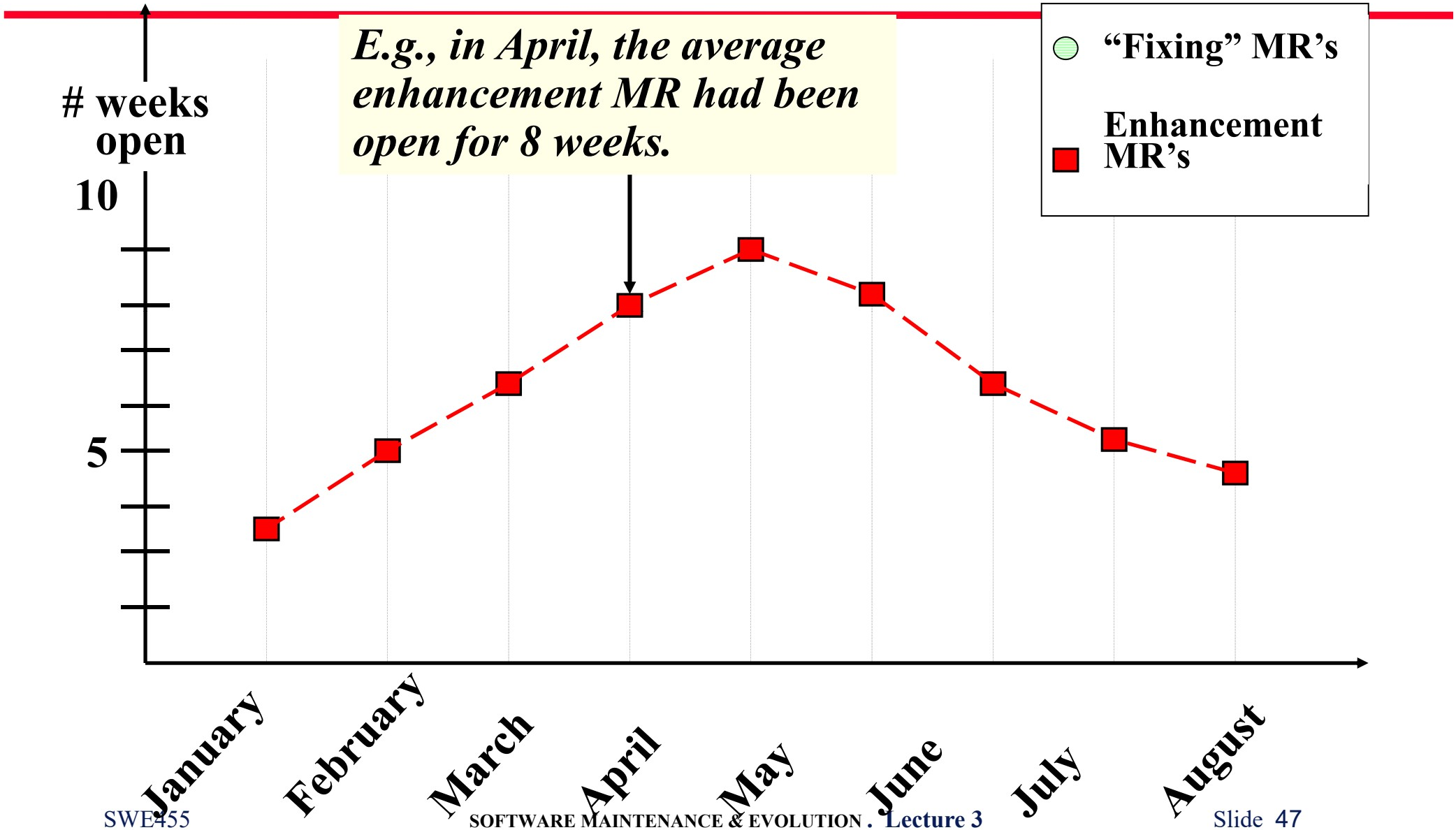
Predicting Relative Maintenance Effort



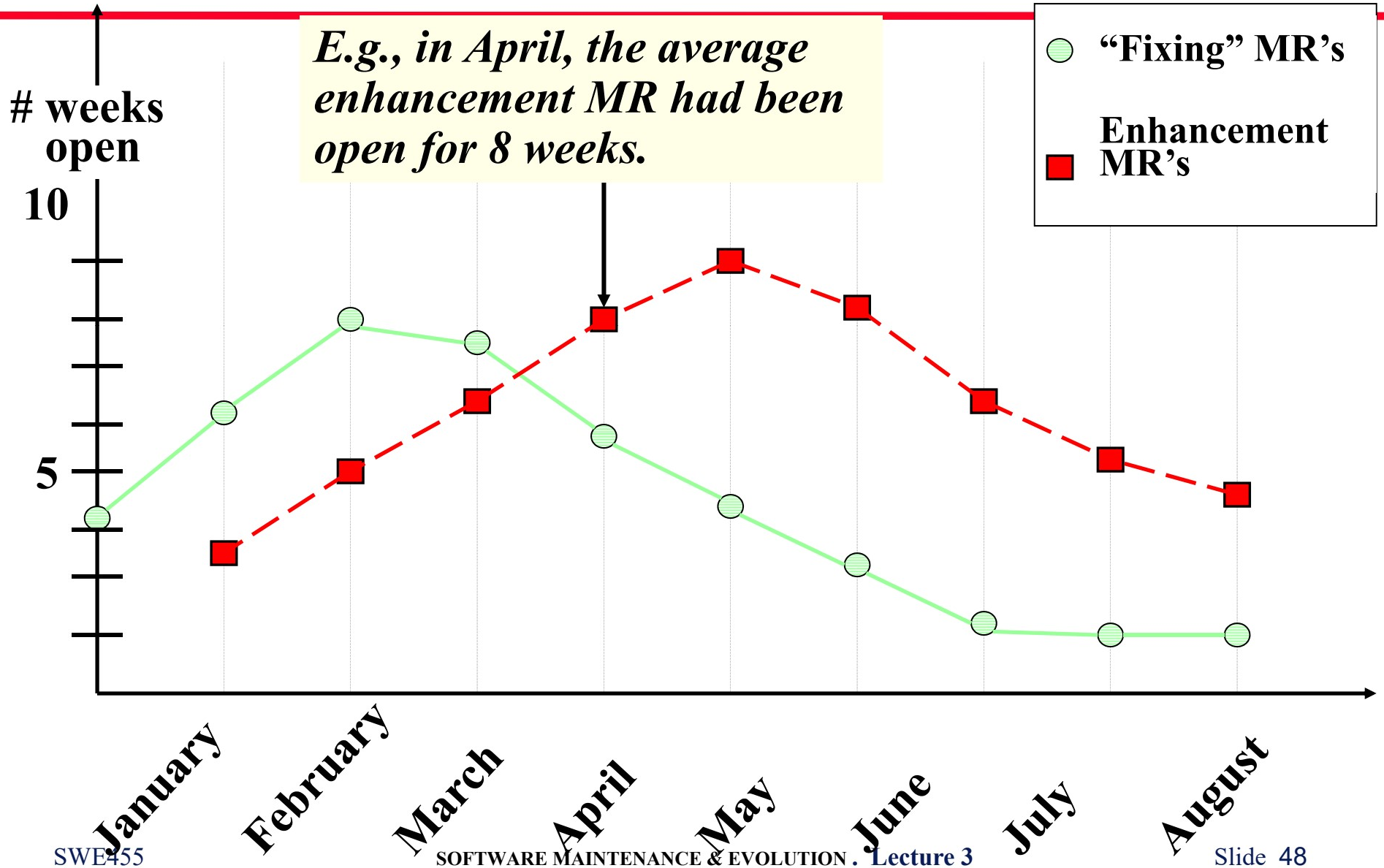
Managing Maintenance
Example profile of "fixing"-type MR's



Profiles of Open Maintenance Requests



Profiles of Open Maintenance Requests



Key points

- Software evolution is the study of the phenomena of system change
- After major empirical study, Lehman proposed that there were a number of ‘laws’ which applied to all systems as they evolved
- Rather than laws, it is more sensible to say ‘observations’. They are shown to be safely applicable in some, not all, circumstances
- Seek to capture by any and all means, the assumptions behind *E*-type software.
- Presents several management challenges
  - manage flow of MR’s
  - motivate personnel
  - ensure all documentation kept up-to-date
- Metrics: plot repairs and enhancements